

Automated Lifecycle for a 3-Tier Web Application

Recipe Rescue

Course: DevOps, gr.4

Student: Rădean Iulian Stelian

Web Application Name: Recipe Rescue

Date: July 21, 2026

Table of Contents

Chapter 1: Project Objectives	3
Chapter 2: Solution Architecture	3
Chapter 3: Technologies stack	5
Chapter 4: Pipeline Implementation	6
Pipeline 1: Validation	6
Pipeline 2: Staging Deployment	6
Pipeline 3: Production Blue-Green Deployment	7
Chapter 5: Monitoring, Self-Healing and Recovery	7
Annexes	8
Annex A: Important Project Files.....	8
Annex B: Main Demo Commands	9
Annex C: Demo Account	10

Chapter 1: Project Objectives

The purpose of this project was to build a complete automated lifecycle for a 3-tier web application. I wanted the final result to be more than a simple application deployed in containers. The goal was to create a realistic DevOps flow where the application can be developed, validated, packaged, deployed, promoted, monitored, backed up, and recovered using modern tools and clear automation.

The application implemented for this project is called Recipe Rescue. It is a practical web application where users can manage pantry ingredients, search for recipes based on available products, add custom recipes, and automatically reduce ingredient quantities after preparing a selected recipe. I chose this idea because it is simple to understand during a live demo, but it still has enough real business logic to make the technical implementation meaningful.

From a DevOps perspective, the main objectives were:

- implement a real 3-tier architecture: frontend, backend, and database;
- run the full application locally with Docker Compose;
- validate code changes through pull request checks;
- build and publish Docker images automatically;
- deploy automatically to a staging environment;
- promote production changes only after validation and approval;
- use Kubernetes as the container orchestration platform;
- use ArgoCD for GitOps-based continuous delivery;
- implement a blue-green deployment strategy for production;
- manage secrets securely outside the repository;
- protect database data through persistence and backups;
- add monitoring and alerting for the deployed environments;
- be able to recreate infrastructure through Terraform if needed.

The project is designed around automation, reliability, traceability, and recovery. Every important step is either stored as code or triggered through a controlled workflow. This makes the project easier to explain, easier to reproduce, and closer to how a real DevOps delivery process would work.

Chapter 2: Solution Architecture

Recipe Rescue follows a classic 3-tier architecture. The first tier is the frontend, built with React and served by Nginx. The second tier is the backend REST API, built with FastAPI. The third tier is PostgreSQL, used for persistent storage.

The logical flow is:

User Browser

-> React frontend served by Nginx

-> /api requests proxied to FastAPI

-> FastAPI reads and writes data in PostgreSQL

The frontend contains multiple pages: home, login/signup, pantry management, recipe listing, recipe finder, recipe details, and recipe creation. The backend exposes endpoints for authentication, ingredients, recipes, matching logic, health checks, and readiness checks. PostgreSQL stores users, pantry items, recipes, recipe ingredients, and user-specific data.

For local development, the application is started with Docker Compose. This creates three containers: one for the frontend, one for the backend, and one for PostgreSQL. The local setup is important because it allows the application to be tested before moving to the cloud deployment flow.

For the cloud infrastructure, the application runs on Amazon EKS. Kubernetes is used to manage containers, services, probes, persistent volumes, namespaces, and deployment strategies. The Kubernetes manifests are organized with a base directory and environment-specific overlays.

The main Kubernetes environments are:

- recipe-rescue-staging: staging frontend, backend, PostgreSQL, and database backup jobs;
- recipe-rescue-production-data: shared production PostgreSQL and production database backups;
- recipe-rescue-production: production frontend, backend, Nginx router, and Argo Rollouts resources.

The production database is intentionally separated from the production application namespace. This makes the blue-green strategy easier to control because both blue and green application versions use the same production database. The production database is not duplicated for every color. This keeps the project simpler, avoids data synchronization problems between two production databases, and matches the current project scope.

Production traffic enters through an AWS LoadBalancer service. That service points to a production Nginx router. Nginx then routes requests to the active frontend and backend services. Argo Rollouts controls which ReplicaSet is active and which ReplicaSet is preview.

The production traffic flow is:

AWS LoadBalancer

- > Production Nginx Router
- > Active frontend service
- > Active backend service
- > Shared production PostgreSQL

ArgoCD is used as the GitOps controller. It watches the Git repository and synchronizes the Kubernetes cluster with the desired state stored in Git. This means Git becomes the source of truth. If someone changes a Kubernetes object manually and that change does not match Git, ArgoCD can detect the drift and restore the desired state.

The high-level software delivery flow is:

Developer branch

- > Pull request to main
- > Automated PR validation
- > Merge to main
- > Docker image publishing
- > Automatic staging deployment
- > Staging live tests
- > Promotion PR to release
- > Release validation and approval
- > ArgoCD production synchronization
- > Argo Rollouts blue-green deployment

This architecture gives a clear separation between development validation, staging deployment, and production release. It also makes the live demo easy to follow because every stage has a visible purpose.

Chapter 3: Technology stack

The project uses the following technology stack:

Area	Technology	Reason
Version Control	GitHub	Used for repository hosting, pull requests, branch protection, and workflow triggers.
CI/CD	GitHub Actions	Used as pipeline-as-code for validation, image publishing, staging deployment, production promotion, infrastructure recovery, and destroy workflows.
Frontend	React, TypeScript, Vite	A modern frontend stack with fast builds, typed code, and good developer experience.
Web Server	Nginx	Serves the frontend static files, proxies API requests, and acts as the production routing layer.
Backend	FastAPI	Provides a clean REST API, automatic documentation, health endpoints, and easy testing.
Database	PostgreSQL	Reliable relational database for persistent application data.
Containers	Docker	Packages frontend and backend in portable images.
Registry	DockerHub	Stores published application images using traceable tags.
Orchestration	Kubernetes on Amazon EKS	Runs and manages containers, services, probes, stateful workloads, and rollouts.
Infrastructure	Terraform	Creates and manages AWS infrastructure and platform services as code.
GitOps	ArgoCD	Synchronizes staging and production environments from Git.
Deployment Strategy	Argo Rollouts	Provides blue-green deployment, preview services, analysis, promotion, and rollback behavior.
Secrets	AWS Secrets Manager and External Secrets Operator	Keeps sensitive values outside Git and injects them into Kubernetes securely.
Backups	AWS S3 Bucket	Stores PostgreSQL backup files for staging, production, and pre-promotion safety backups.
Monitoring	Prometheus, Grafana, Blackbox Exporter, Alertmanager	Provides metrics, dashboards, endpoint checks, and alert routing.
Email Alerts	AWS SES SMTP	Sends Alertmanager notifications by email.

The final stack is still aligned with the DevOps objective of the project because all delivery stages are automated and all important infrastructure/deployment definitions are stored as code.

Chapter 4: Pipeline Implementation

Pipeline 1: Validation

Pipeline 1 is the continuous integration pipeline. It runs on pull requests targeting the main branch. The goal of this pipeline is to validate changes before they are merged.

The main workflow is called PR Checks. It does not contain all logic in one large file. Instead, it calls separate check workflows. This makes failures easier to understand in GitHub because every important validation has its own job name.

The validation pipeline includes:

- backend dependency consistency;
- backend linting;
- backend tests;
- frontend dependency audit;
- frontend linting;
- frontend production build;
- backend Docker image build;
- frontend Docker image build;
- Kubernetes manifest rendering;
- Kubernetes schema validation;
- Kubernetes linting;
- custom Kubernetes policy checks;
- ArgoCD manifest policy checks;
- pull request status comment.

The backend tests validate the application logic, especially recipe matching behavior. The frontend checks validate dependency safety, linting, and production build correctness. The Kubernetes checks validate that manifests can be rendered and that they respect basic project rules, such as not committing real Kubernetes Secrets and having required probes/resources.

If one required check fails, GitHub blocks the merge. This protects the main branch and makes sure that only validated changes can move forward.

Pipeline 2: Staging Deployment

Pipeline 2 starts after changes are merged into main.

The first workflow is Docker Publish. It builds the frontend and backend Docker images and pushes them to DockerHub. Each image receives three tags:

```
latest
sha-<12-character-commit-sha>
build-<github-run-number>
```

The sha-... tag is the most important for deployments because it points to an exact commit. This makes rollback and tracking much easier. The build-... tag is easier to read during demonstrations. The latest tag always points to the newest image.

After Docker publishing succeeds, the staging deployment workflow updates the staging branch with the new image tag. ArgoCD watches the staging branch and automatically deploys the updated manifests to the staging namespace.

After the staging deployment, a live test workflow validates the real deployed environment. It checks:

- frontend availability;
- backend health endpoint through the frontend proxy;
- backend readiness endpoint and database connectivity;
- signup/authentication flow;
- recipes API availability.

This staging pipeline proves that the application does not only build correctly, but also runs correctly in Kubernetes.

Pipeline 3: Production Blue-Green Deployment

Pipeline 3 is the production deployment pipeline. It starts with a manual promotion workflow that opens a pull request from main to release.

The release branch represents the approved production state. It is not updated directly by normal development work. Before production is updated, the release PR must pass deployment-focused checks and receive approval.

After validation and approval, the completion workflow updates the release branch to the approved commit and pins production manifests to the matching Docker image tag. ArgoCD then detects the change on release and starts synchronizing production.

Production uses a blue-green strategy with Argo Rollouts. The idea is that the current version continues serving users while the new version is deployed separately as a preview version.

The production rollout flow is:

1. ArgoCD detects a change in the release branch.
2. Argo Rollouts creates a preview ReplicaSet for the new version.
3. The backend rollout runs a pre-promotion PostgreSQL backup to S3.
4. Backend preview smoke tests are executed.
5. The frontend rollout waits until the backend rollout is healthy.
6. Frontend preview smoke tests are executed.
7. If all checks pass, Argo Rollouts switches the active services to the new ReplicaSet.
8. Nginx continues routing traffic to the active services, so the public URL stays the same.
9. The previous version is kept temporarily for rollback protection.
10. Post-promotion checks confirm that the new active version is healthy.

If the pre-promotion or post-promotion checks fail, the unsafe version is not promoted or the rollout is aborted. This protects production from broken releases and makes the deployment process safer.

Chapter 5: Monitoring, Self-Healing and Recovery

The project includes a minimal monitoring stack deployed by Terraform. The monitoring stack contains Prometheus, Grafana, Blackbox Exporter, and Alertmanager.

Prometheus collects metrics from Kubernetes and evaluates alert rules. Blackbox Exporter is used for HTTP checks. Prometheus asks Blackbox Exporter to test important application URLs from inside the cluster. This confirms whether staging and production endpoints are reachable.

The monitored endpoints include:

- staging frontend;
- staging backend readiness endpoint;
- production frontend through the production router;
- production backend readiness endpoint through the production router.

Grafana is used to visualize the system. It contains a Recipe Rescue dashboard that shows endpoint health, running pods, pod restarts, database readiness, and controller visibility.

Alertmanager receives alerts from Prometheus. When email alerts are enabled, Alertmanager sends notifications through AWS SES SMTP. This proves that the project has not only dashboards, but also active alerting.

The main alert rules are:

- RecipeRescueEndpointDown: fires when an endpoint is down for more than 2 minutes;
- RecipeRescuePodRestarting: fires when a Recipe Rescue pod restarted recently.

The project also has several self-healing and recovery mechanisms:

- Kubernetes liveness probes can restart unhealthy containers.
- Kubernetes readiness probes prevent unhealthy pods from receiving traffic.
- ArgoCD self-healing restores resources that drift from Git.
- Argo Rollouts blocks or aborts unsafe production releases.
- Terraform can recreate the AWS/EKS infrastructure.
- PostgreSQL backups in S3 support database recovery.

For data protection, PostgreSQL uses persistent volumes during normal runtime and S3 backups for disaster recovery. The S3 backup bucket is kept outside the destroyable EKS stack. This means the cluster can be destroyed and rebuilt while backup files remain available.

Database backups are created for:

- staging;
- production;
- production pre-promotion safety backups.

The infrastructure recovery workflow can rebuild the AWS environment. After the cluster is recreated, ArgoCD redeploys the application from Git, and database restore jobs can bring back data from S3 backups. This gives the project a complete recovery story, not only an application deployment story.

Annexes

Annex A: Important Project Files

File or Directory	Purpose
<code>`.github/workflows/pr-checks.yml`</code>	Main pull request validation workflow.
<code>`.github/workflows/check-*.yml`</code>	Individual reusable validation checks.
<code>`.github/workflows/docker-publish.yml`</code>	Builds and publishes Docker images.
<code>`.github/workflows/deploy-staging.yml`</code>	Updates staging deployment state after main changes.
<code>`.github/workflows/staging-live-tests.yml`</code>	Tests the live staging environment.
<code>`.github/workflows/promote-release.yml`</code>	Opens the production promotion pull request.
<code>`.github/workflows/complete-release-promotion.yml`</code>	Completes approved production promotion.
<code>`.github/workflows/infrastructure-recovery.yml`</code>	Recreates or repairs AWS infrastructure using Terraform.
<code>`.github/workflows/infrastructure-destroy.yml`</code>	Destroys AWS infrastructure for cost control.
<code>`backend/`</code>	FastAPI REST API source code.
<code>`frontend/`</code>	React frontend source code.
<code>`k8s/base/`</code>	Shared Kubernetes manifests.
<code>`k8s/overlays/staging/`</code>	Staging Kubernetes overlay.
<code>`k8s/overlays/production/`</code>	Production application overlay with Rollouts and

	Nginx router.
`k8s/overlays/production-data/`	Shared production PostgreSQL overlay.
`argocd/`	ArgoCD app-of-apps configuration.
`infra/terraform/aws/`	AWS infrastructure, EKS, platform services, monitoring, and recovery.

Annex B: Main Demo Commands

Run the application locally:

```
docker compose up --build
```

Connect to the EKS cluster:

```
aws eks update-kubeconfig --region eu-central-1 --name recipe-rescue-shared-eks
```

Open ArgoCD:

```
kubectrl port-forward svc/argocd-server -n argocd 8080:443
```

Open Argo Rollouts dashboard:

```
kubectrl port-forward svc/argo-rollouts-dashboard -n argo-rollouts 3100:3100
```

Open Grafana:

```
kubectrl port-forward svc/monitoring-grafana -n monitoring 3000:80
```

Open Alertmanager:

```
kubectrl port-forward svc/monitoring-alertmanager -n monitoring 9093:9093
```

Trigger a controlled monitoring demo alert:

```
@'
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: recipe-rescue-demo-alert
  namespace: monitoring
spec:
  groups:
    - name: recipe-rescue.demo
      rules:
        - alert: RecipeRescueDemoAlert
          expr: vector(1)
          for: 30s
          labels:
            severity: warning
            environment: demo
            component: monitoring
          annotations:
            summary: Recipe Rescue demo alert
            description: Demo alert proving Prometheus to Alertmanager to email.
'@ | kubectl apply -f -
```

Remove the demo alert:

```
kubectrl delete prometheusrule recipe-rescue-demo-alert -n monitoring
```

Annex C: Demo Account

Email: demo@reciperescue.local

Password: demo123

This account contains seeded pantry data and can be used for a quick application walkthrough.